25DCE026

Department of Computer Engineering

Practical - 14

Practical Title: Admin Dashboard with Role-Based Access Control, Dynamic Menu, and Audit Log

Problem Definition: Develop an admin dashboard that displays user statistics, student count, event count, and latest registrations. Restrict access to admin users only and show dynamic menu options based on user role.

Theory :

1. Are dashboard values loaded dynamically from the database?

Yes. KPI counts (Total Users, Active Students, Total Events, Event Registrations) are aggregated using SQL `COUNT()` queries and displayed in real-time summary cards.

2. Is access restricted to admin users?

Yes. `require_role('admin')` enforces RBAC at the page controller level. Unauthorized access triggers an immediate redirect to the login screen with a `forbidden` message.

3. Are role-based menus working?

Yes. Navigation is rendered dynamically by `render_dynamic_menu()` based on the user's session role. Admins see full administrative options; students see a simplified menu.

4. Are admin actions recorded in audit_logs?

Yes. Status toggles, student edits, event changes, and login events are all logged to `audit_logs` with user ID, action code, description, IP address, and timestamp.

5. What Intermediate and Advanced extensions are provided?

Intermediate: Active/Inactive user toggle (`toggle_user.php`) with self-deactivation protection. **Advanced:** Interactive Chart.js bar chart visualizing event registration distribution.

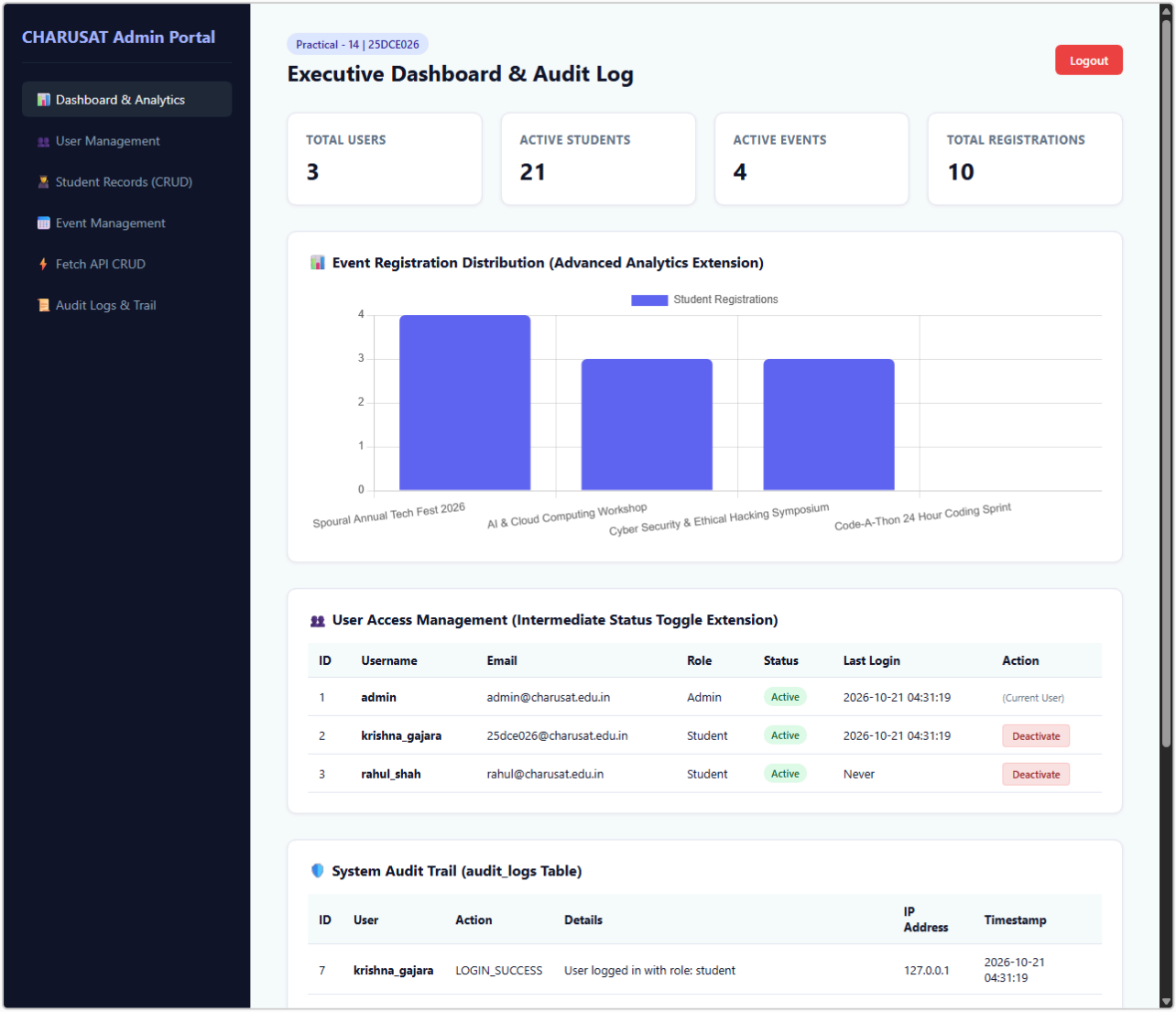
Implementation / Output / Result

```
require_role('admin');  // RBAC Middleware Guard
$total_users       = $conn->query("SELECT COUNT(*) AS c FROM users")->fetch_assoc()['c'];
$active_students   = $conn->query("SELECT COUNT(*) AS c FROM students WHERE status='active'")-
>fetch_assoc()['c'];
$total_events      = $conn->query("SELECT COUNT(*) AS c FROM events")->fetch_assoc()['c'];
$total_registrations = $conn->query("SELECT COUNT(*) AS c FROM event_registrations")-
>fetch_assoc()['c'];
render_dynamic_menu('dashboard');
```

25DCE026

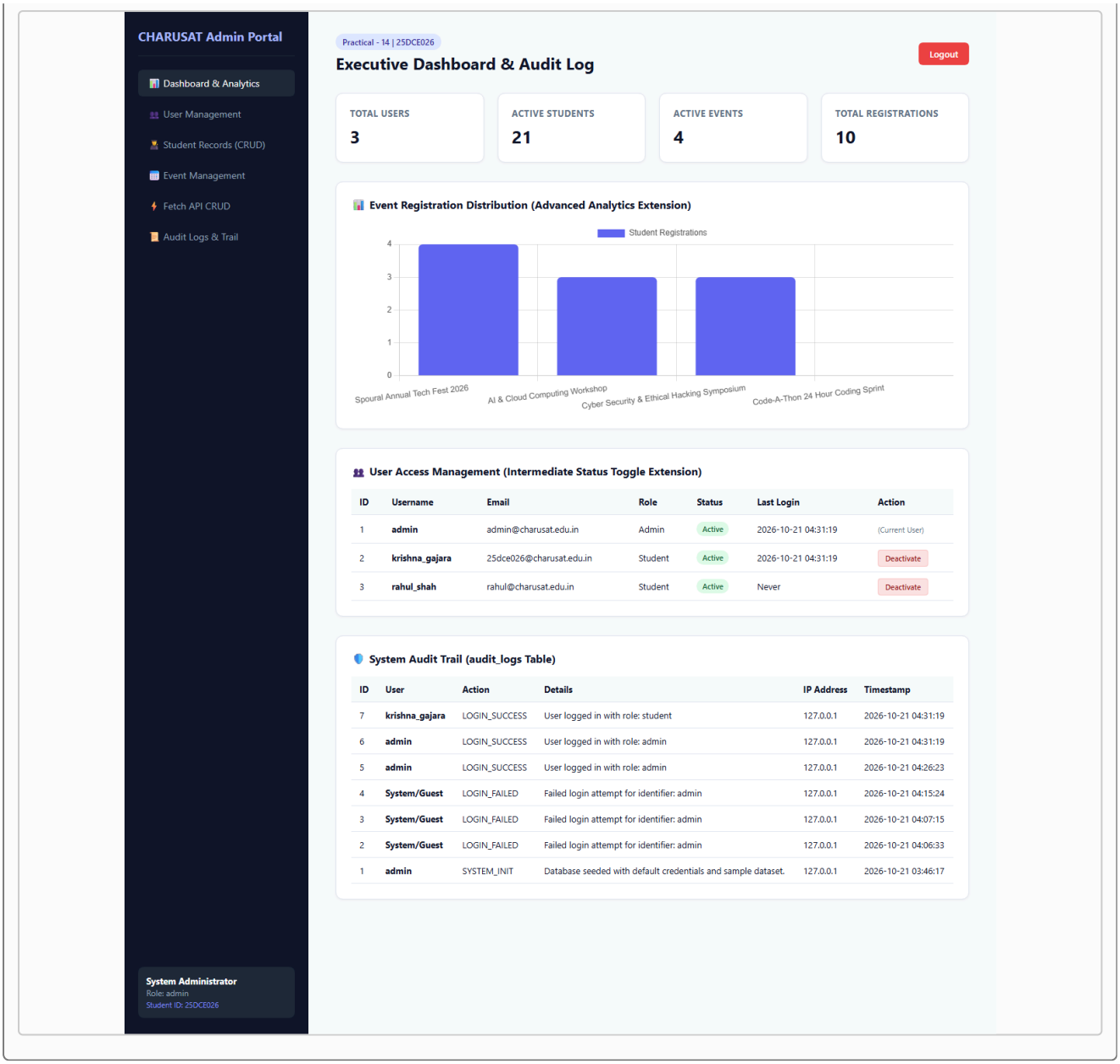
Output 1: Executive Admin Dashboard — KPI Cards, Chart.js Analytics & User Management

Live browser screenshot: Dashboard with metric cards (3 Users, 21 Students, 4 Events, 10 Registrations), Event Registration bar chart, User status toggle table.



Output 2: System Audit Trail — Full Audit Log Table with IP Addresses

Live browser screenshot: Full-page view showing System Audit Trail table with action codes (LOGIN_SUCCESS, SESSION_TIMEOUT), IP 127.0.0.1, and timestamps.



GitHub Repository Link:
https://github.com/25dce026-krishna/StudentHub

Learning Outcomes:

- Built an administrative dashboard with SQL-driven analytical KPI aggregations.
- Enforced Role-Based Access Control (RBAC) to protect sensitive administrative routes.
- Generated dynamic role-tailored navigation structures using PHP helper functions.
- Established an immutable audit logging pipeline capturing action traces and client IP addresses.
- Integrated Chart.js client-side visual analytics with live database aggregations.
- Implemented user account lifecycle controls with status toggles and self-deactivation protection.